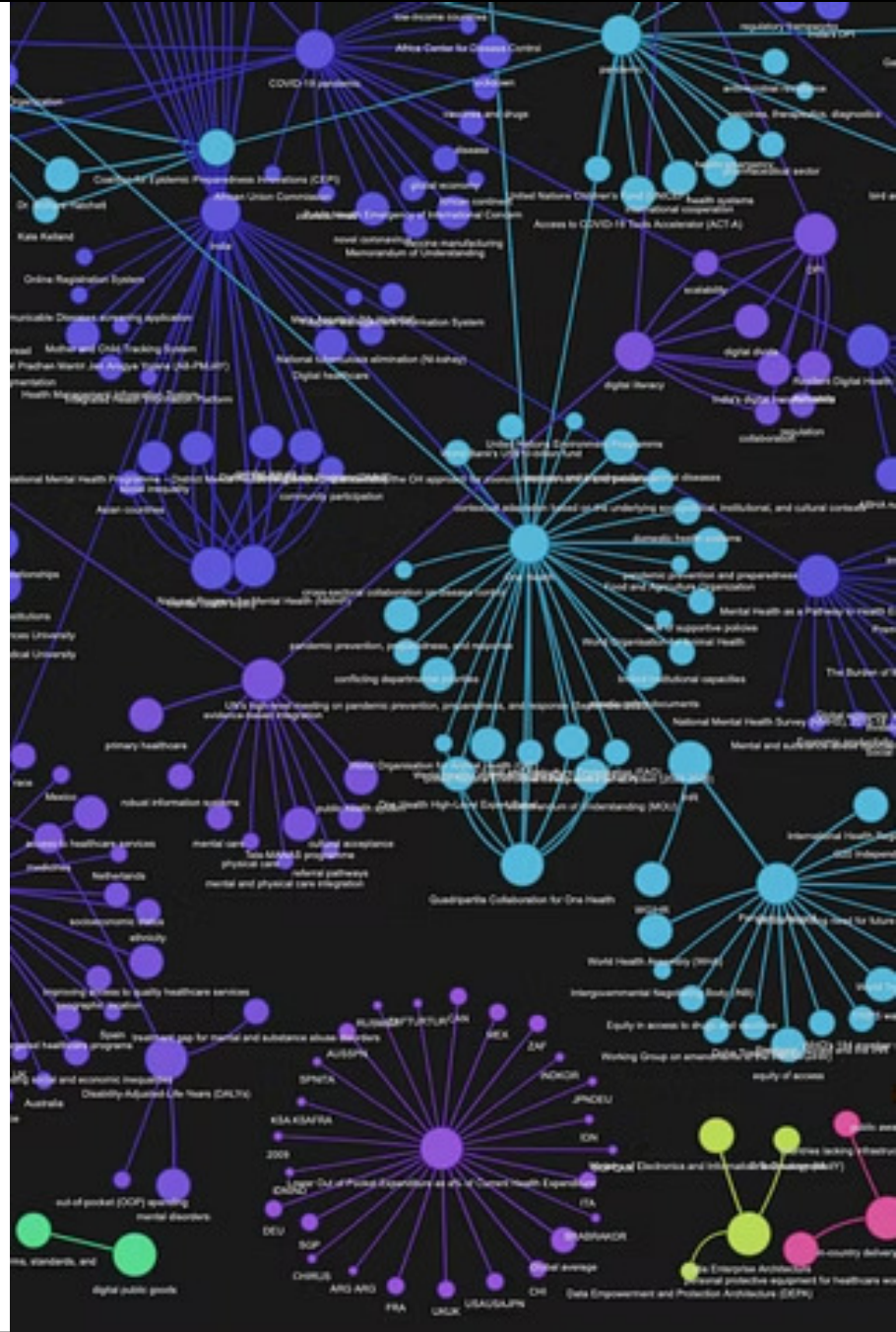
Machine Translation & Sequence-to-Sequence Models

SESSION 13-14

60 MINUTES EACH

A deep dive into the evolution, architecture, and challenges of automatically translating human language using neural networks



SESSION OVERVIEW

What We'll Cover Today

01

Machine Translation Fundamentals

Understanding the goal and complexity of automatic translation systems

02

Historical Evolution

From rule-based systems to statistical approaches and neural networks

03

Seq2Seq Architecture

Deep dive into encoders, decoders, and attention mechanisms

04

Evaluation & Modern Challenges

BLEU scores, idioms, low-resource languages, and practical applications

The Machine Translation Challenge

Machine Translation aims to automatically convert text from a source language to a target language while preserving both meaning and fluency. This seemingly simple goal masks extraordinary complexity.

MT is considered a "complete" NLP task because successful translation requires solving nearly every other NLP problem simultaneously: word sense disambiguation, syntactic parsing, morphological analysis, semantic understanding, and natural language generation.

HISTORICAL CONTEXT

The Evolution of Machine Translation

1

1950s-1980s: Rule-Based MT

Hand-crafted grammars and bilingual dictionaries dominated the landscape

2

1990s-2000s: Statistical MT

Data-driven approaches using parallel corpora and probability models emerged

3

Mid-2010s: Neural MT

Deep learning revolutionized translation with sequence-to-sequence architectures

4

Present: Transformer Era

Attention mechanisms and pre-trained models achieve near-human quality

Rule-Based Machine Translation

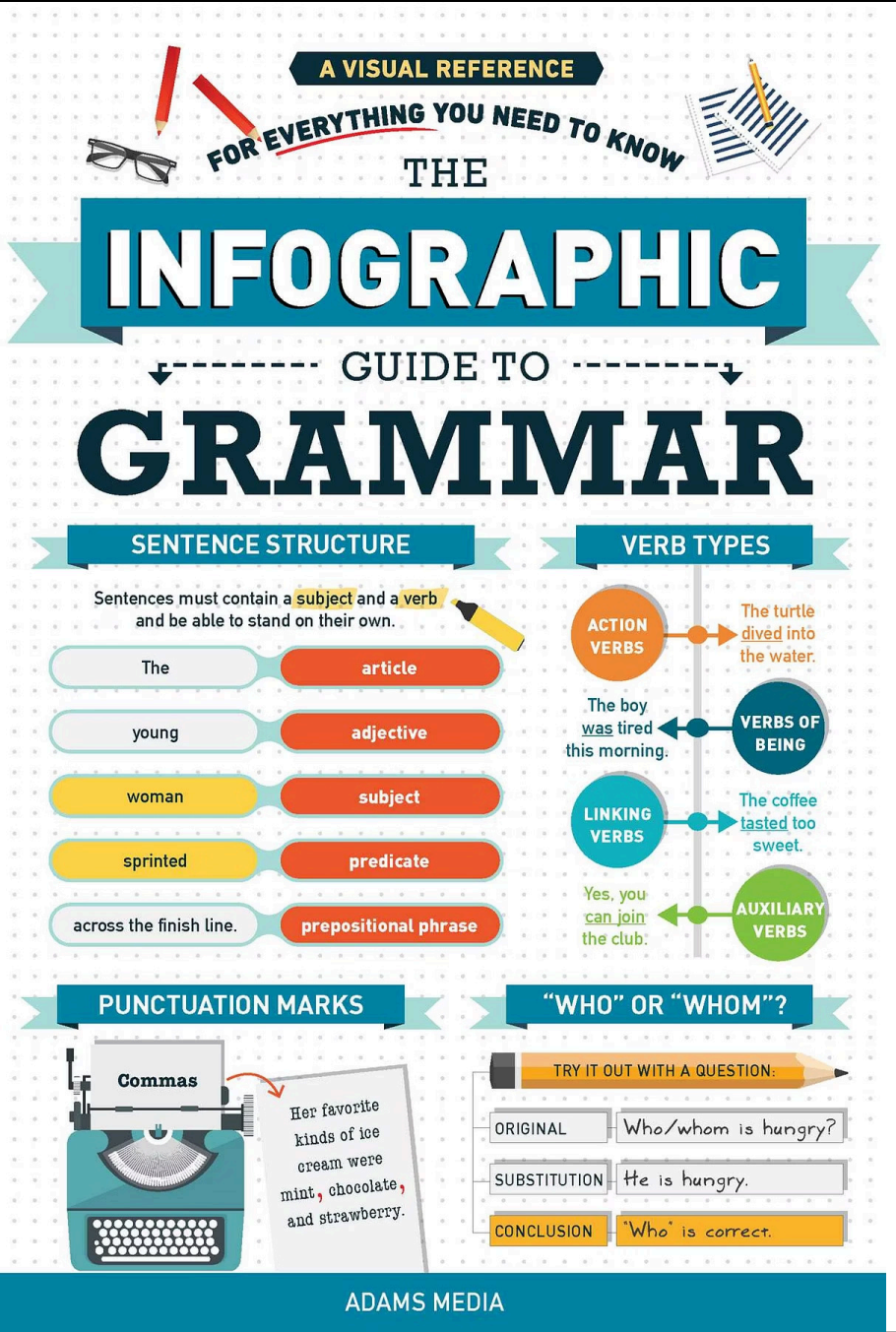
The Pioneer Era: 1950s-1980s

The Approach

Massive bilingual dictionaries paired with hand-crafted transformation rules

RBMT systems relied on linguists manually encoding grammar rules for each language pair. For example: "If a Noun is followed by an Adjective in English, swap their positions in French."

These systems required years of expert knowledge to build and maintain, making them expensive and inflexible.



The Brittleness Problem

Rule-based systems were fundamentally brittle—they broke easily when encountering unexpected input. Natural language is filled with exceptions, idioms, slang, neologisms, and complex nested structures that defied rigid rules.

Exception Overload

Every rule needed sub-rules for exceptions, creating exponential complexity

Poor Scalability

Adding a new language pair required starting from scratch

Limited Adaptation

Unable to learn from mistakes or improve over time

DATA-DRIVEN ERA

Statistical Machine Translation

The Paradigm Shift: 1990s-2000s

SMT marked a fundamental shift from human-crafted rules to learning from data. Instead of encoding linguistic knowledge manually, these systems extracted translation patterns from **parallel corpora**—large collections of documents professionally translated by humans.

Sources included UN proceedings, Canadian Parliament records, and the European Parliament corpus, providing millions of sentence pairs for training.

The Noisy Channel Model

SMT reframed translation as a decoding problem inspired by information theory. The model assumes the target language text was "corrupted" through a noisy channel into the source language.

1

Translation Model P(f|e)

Ensures words are good matches for the source text

2

Language Model P(e)

Ensures output sounds like natural human language

3

Argmax Decoding

Find the best translation maximizing both probabilities

Phrase-Based Statistical MT

The Method

The source sentence is segmented into small chunks (phrases). The system consults learned probability tables to find the most likely translation for each phrase.

These probabilities are extracted automatically from millions of aligned sentence pairs in the training corpus.

Phrase Extraction

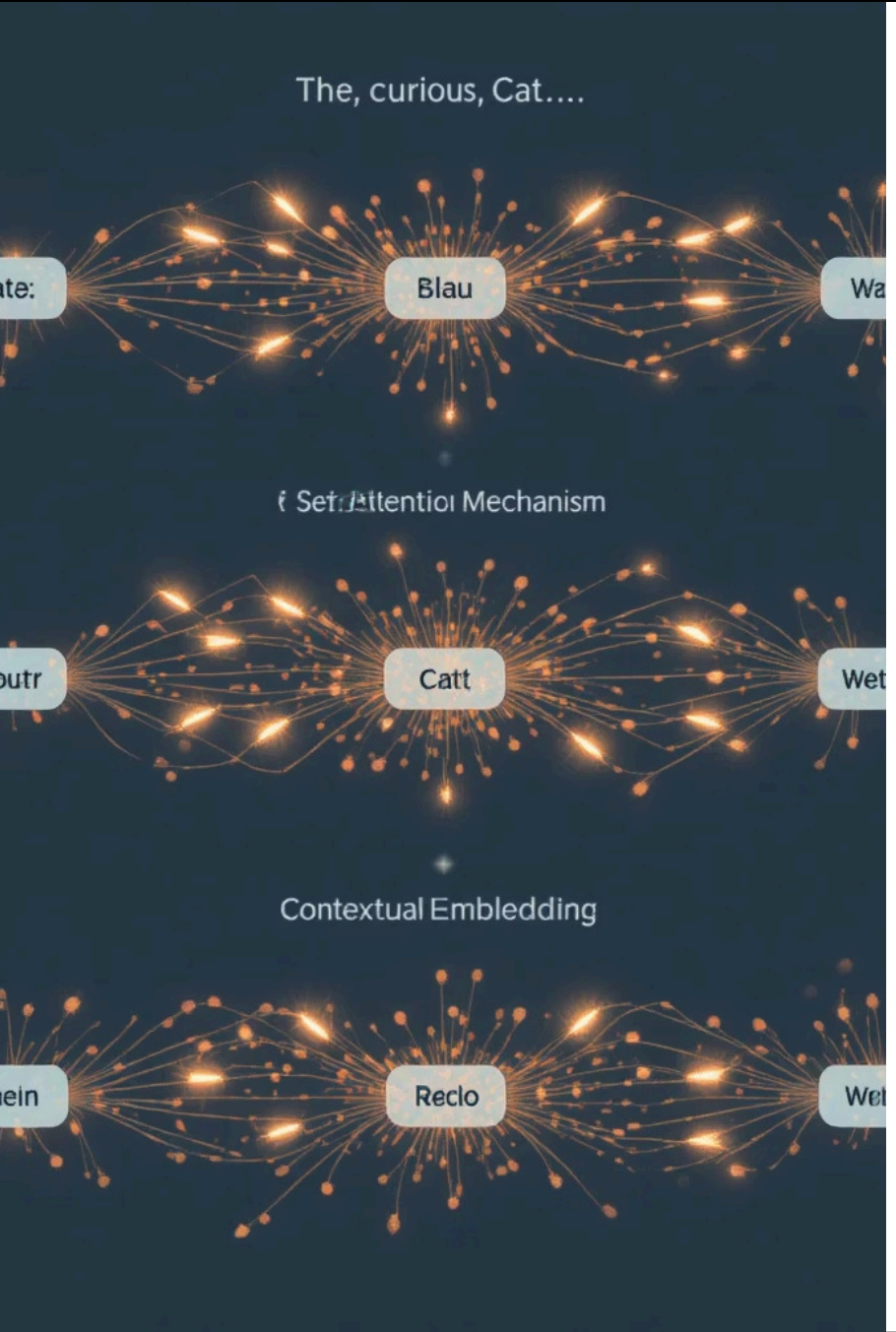
Identify recurring multi-word patterns in parallel text

Probability Tables

Calculate translation likelihoods from aligned data

Decoding Search

Find optimal phrase combination using beam search



The Neural Revolution

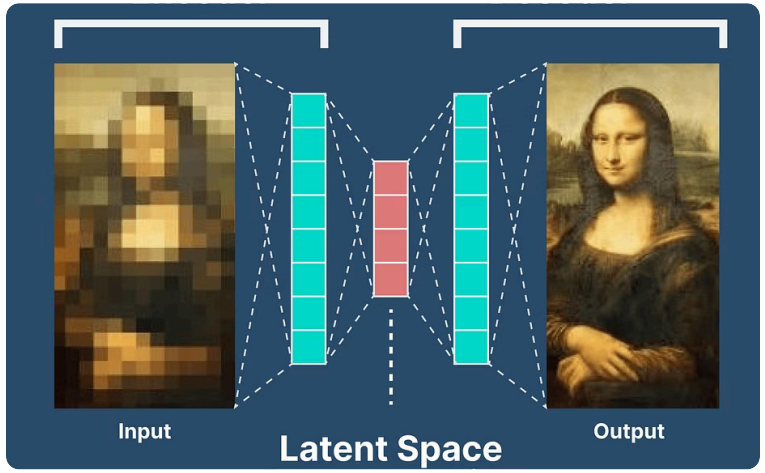
Mid-2010s to Present

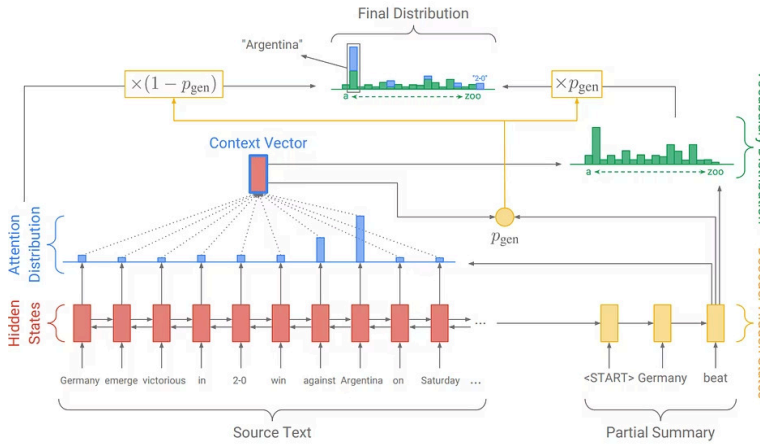
Neural Machine Translation (NMT) transformed the field by replacing discrete probability tables with continuous vector representations. Since 2015, Sequence-to-Sequence neural networks have become the dominant paradigm.

This shift enabled end-to-end learning where a single neural network learns all aspects of translation simultaneously, from word meanings to syntax to discourse structure.

CORE ARCHITECTURE

Sequence-to-Sequence Models





The Encoder

A neural network (RNN, LSTM, or Transformer) reads the source sentence word by word and compresses it into a dense mathematical vector representation.

The Decoder

Another neural network takes the encoded vector and generates the target sentence one word at a time, conditioned on previously generated words.

The Encoder in Detail

The encoder processes the source sentence sequentially, building up a representation that captures both the meaning of individual words and their relationships within the sentence.

Word Embeddings

Convert each word to a dense vector representation

Sequential Processing

RNN or Transformer layers capture context and dependencies

Final State

Produce a fixed-size vector encoding the entire sentence

The Decoder in Detail

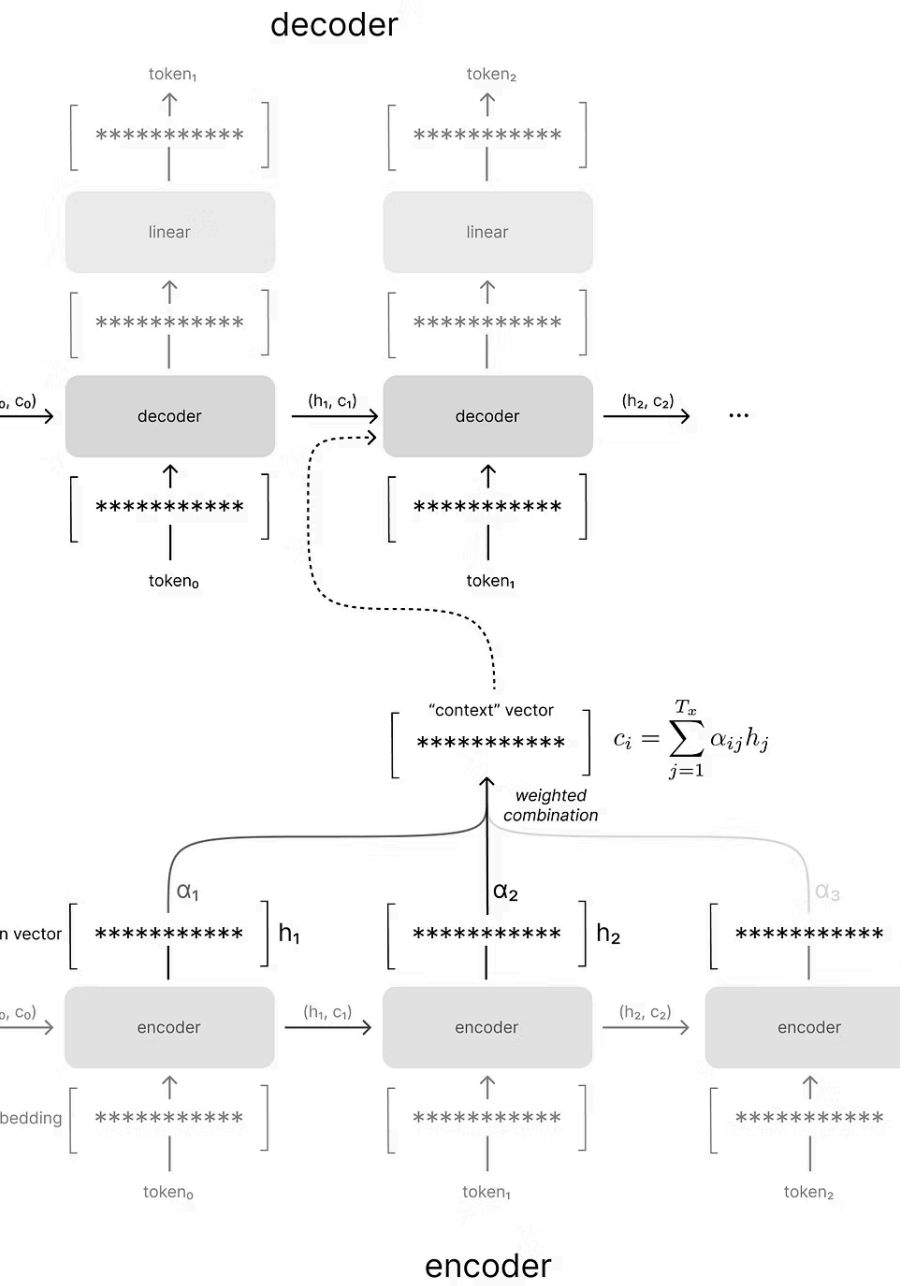
The decoder generates the target sentence autoregressively—each word depends on all previously generated words. This creates fluent, coherent output that respects target language grammar.

Generation Process

1. Start with a special "begin" token
2. Use encoder vector plus previous words
3. Predict probability distribution over vocabulary
4. Sample or select the most likely word
5. Repeat until "end" token generated

Key Mechanisms

The decoder maintains its own hidden state that evolves as generation proceeds. Teacher forcing during training uses ground-truth words instead of predictions to stabilize learning.



The Attention Breakthrough

The original Seq2Seq architecture had a critical bottleneck: compressing an entire sentence into a single fixed-size vector lost important information, especially for long sentences.

The **attention mechanism**, introduced in 2014 by Bahdanau et al., revolutionized neural MT by allowing the decoder to dynamically focus on different parts of the source sentence when generating each target word.

How Attention Works

Compute Alignment

Calculate relevance scores between current decoder state and each encoder state

Create Weights

Apply softmax to scores, producing attention weights that sum to 1.0

Weighted Sum

Multiply encoder states by weights and sum to create context vector

Generate Word

Concatenate context with decoder state to predict next word

Attention: A Concrete Example

Consider translating English "Red car" to French "Voiture rouge" (literally "Car red"). English places adjectives before nouns, while French places them after.

1

Generate "Voiture"

Attention focuses strongly on "car" in the source

2

Generate "rouge"

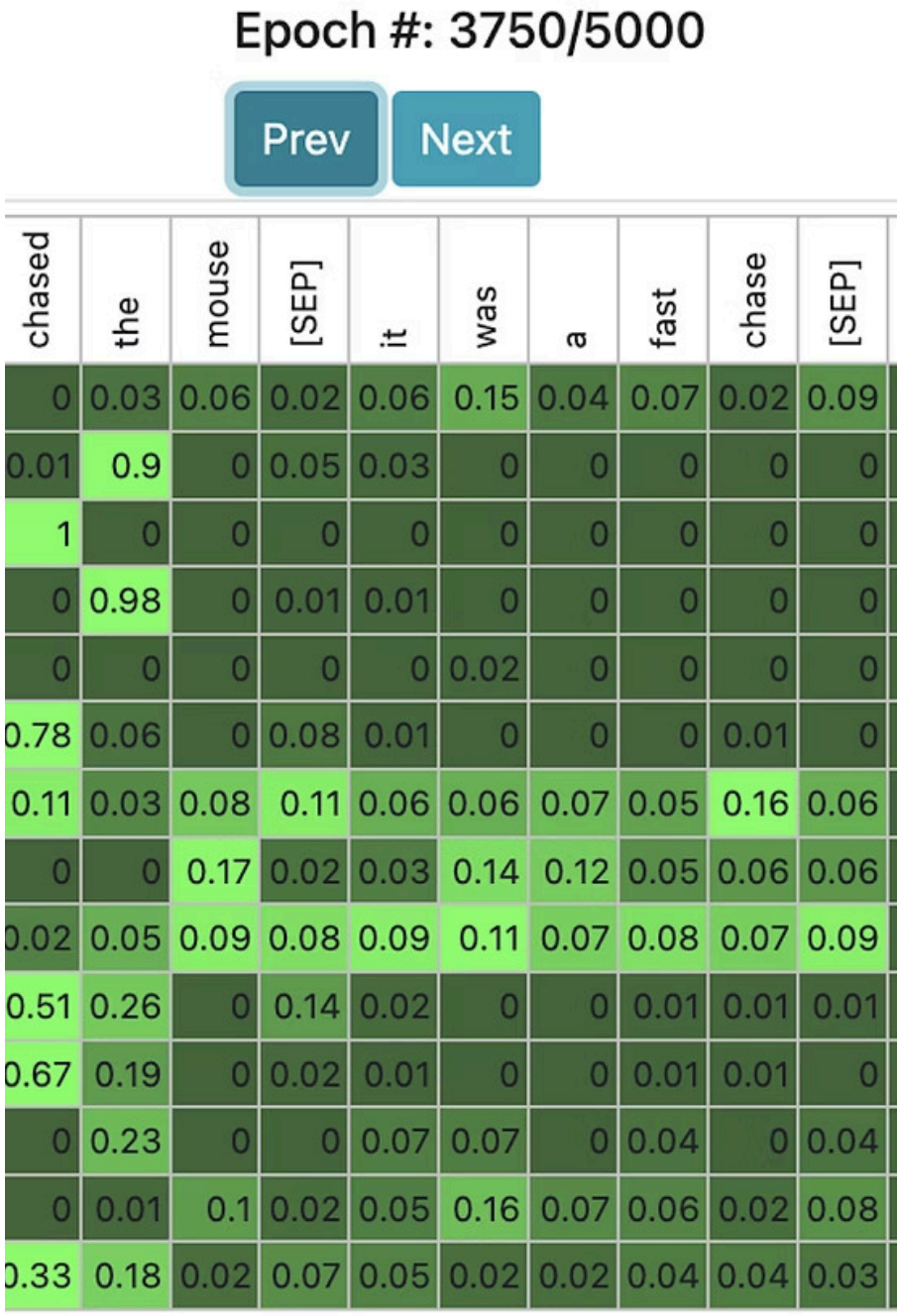
Attention shifts to focus on "red" in the source

The model learns this reordering pattern automatically from data, without any hand-written rules about adjective placement.

Attention Visualization

Attention weights can be visualized as heatmaps showing which source words the model focuses on when generating each target word. These visualizations reveal that the model learns linguistically meaningful alignments.

For complex phenomena like long-distance dependencies, clause reordering, or handling pronouns, attention patterns show the model has implicitly learned syntactic and semantic relationships without explicit grammatical rules.



EVALUATION

Measuring Translation Quality

The BLEU Score Challenge

Evaluating MT is notoriously difficult because there are many correct ways to translate any sentence. Automated metrics try to approximate human judgment by comparing machine output to reference translations.

The industry standard is **BLEU** (Bilingual Evaluation Understudy), which measures n-gram overlap between candidate and reference translations.

How BLEU Works

Core Idea

Count how many 1-grams, 2-grams, 3-grams, and 4-grams in the machine translation appear in human reference translations

Precision-Based

Rewards matching phrases from reference

Brevity Penalty

Penalizes translations shorter than reference

0-100 Scale

Higher scores indicate better quality

BLEU Limitations

While BLEU provides a quick, automatic evaluation metric that correlates reasonably well with human judgments, it has significant drawbacks that practitioners must understand.

Misses Synonyms

"Happy" vs "joyful" scores as completely different despite identical meaning

Ignores Fluency

Grammatically broken text can score high if words match reference

Poor for Single Sentences

Works best on corpus-level evaluation, unreliable for individual examples

Reference Dependency

Requires high-quality human translations as gold standards

Challenge: Idioms and Figurative Language

Idioms present one of the most difficult challenges for MT systems. Consider the English idiom "kick the bucket," meaning to die.

A literal translation to another language would describe striking a pail with one's foot—completely missing the figurative meaning. Neural models must learn which phrases are idiomatic and find equivalent expressions in the target language, not word-for-word translations.

Challenge: Low-Resource Languages

Modern neural MT systems are data-hungry, requiring millions of parallel sentence pairs to achieve good performance. This creates a stark divide between high-resource and low-resource languages.

1B+

English-French

Parallel sentences available from European institutions, web crawls, and commercial sources

100K

English-Swahili

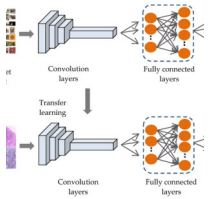
Limited parallel data available, primarily religious texts and news

1K

English-Luganda

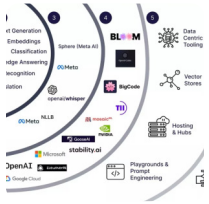
Extremely limited resources, research-stage only

Approaches for Low-Resource MT



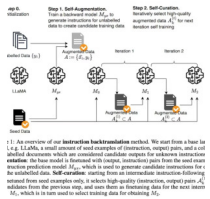
Transfer Learning

Pre-train on high-resource language pairs, then fine-tune on low-resource target with limited data



Multilingual Models

Train single model on many language pairs simultaneously, enabling knowledge sharing across languages



Back-Translation

Generate synthetic parallel data by translating monolingual target text back to source

Challenge: Out-of-Vocabulary Words

Traditional word-based models struggle when encountering words never seen during training—named entities, technical jargon, neologisms, or rare morphological forms.

The Problem

A fixed vocabulary creates a hard boundary. Unknown words are replaced with a generic UNK token, losing critical information.

Modern Solutions

- Subword tokenization (BPE, WordPiece)
- Character-level models
- Hybrid architectures combining both
- Copy mechanisms that can reproduce source words

Modern MT in Practice

Today's MT systems achieve remarkable quality for high-resource language pairs, approaching human parity on some benchmarks. Leading implementations include:

Google Translate

Multilingual Transformer models serving 100+ languages to billions of users daily

OpenNMT

Open-source toolkit for researchers and practitioners to build custom MT systems

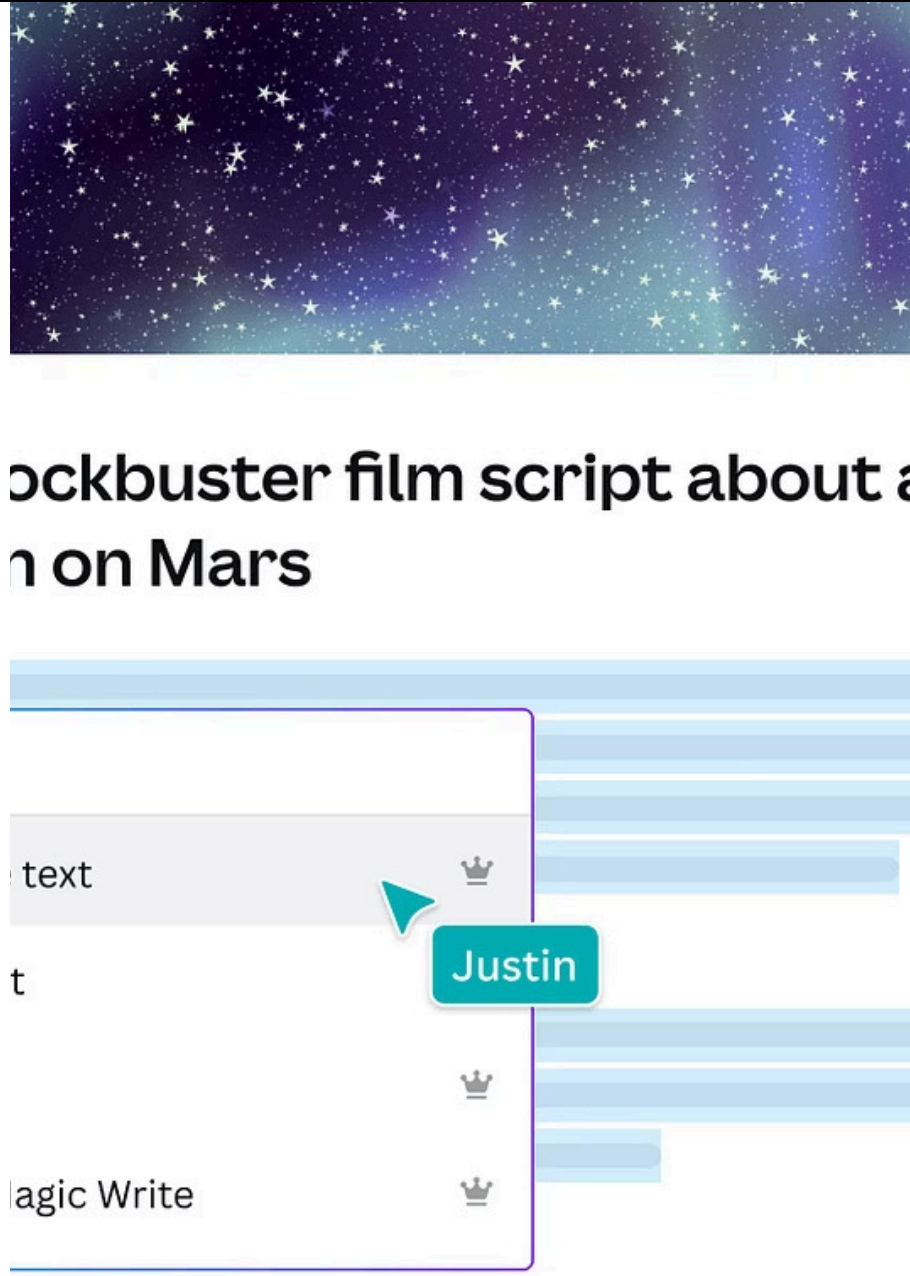
Hugging Face

Pre-trained models like MarianMT accessible via simple Python APIs

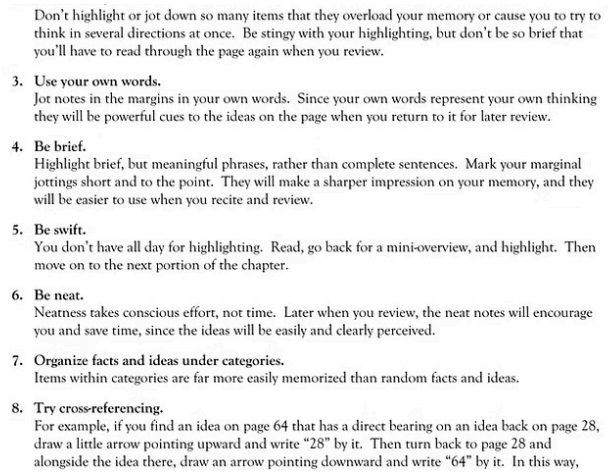
SESSION 14

Text Summarization

Automatic text summarization distills lengthy documents into shorter versions while preserving the most important information. This fundamental NLP task has applications in news aggregation, research literature review, meeting notes, and content curation.



Two Approaches to Summarization



Don't highlight or jot down so many items that they overload your memory or cause you to try to think in several directions at once. Be stingy with your highlighting, but don't be so brief that you'll have to read through the page again when you review.

3. **Use your own words.**
   Jot notes in the margins in your own words. Since your own words represent your own thinking they will be powerful cues to the ideas on the page when you return to it for later review.
4. **Be brief.**
   Highlight brief, but meaningful phrases, rather than complete sentences. Mark your marginal jottings short and to the point. They will make a sharper impression on your memory, and they will be easier to use when you recite and review.
5. **Be swift.**
   You don't have all day for highlighting. Read, go back for a mini-overview, and highlight. Then move on to the next portion of the chapter.
6. **Be neat.**
   Neatness takes conscious effort, not time. Later when you review, the neat notes will encourage you and save time, since the ideas will be easily and clearly perceived.
7. **Organize facts and ideas under categories.**
   Items within categories are far more easily memorized than random facts and ideas.
8. **Try cross-referencing.**
   For example, if you find an idea on page 64 that has a direct bearing on an idea back on page 28, draw a little arrow pointing upward and write "28" by it. Then turn back to page 28 and alongside the idea there, draw an arrow pointing downward and write "64" by it. In this way,

Extractive Summarization

Select and copy the most important sentences from the source document verbatim—like using a highlighter on a textbook

Abstractive Summarization

Generate new sentences that capture key ideas in novel phrasings—like a human writing summary notes

EXTRACTIVE METHODS

Extractive Summarization Techniques

Extractive methods rank sentences by importance and select the top-k. This approach is simpler, more reliable, and produces guaranteed factual accuracy since sentences come directly from the source.

1

Frequency-Based

Sentences containing high-frequency content words are likely important

2

Position-Based

In news articles, the first sentences typically contain the "lead" summarizing who, what, when, where

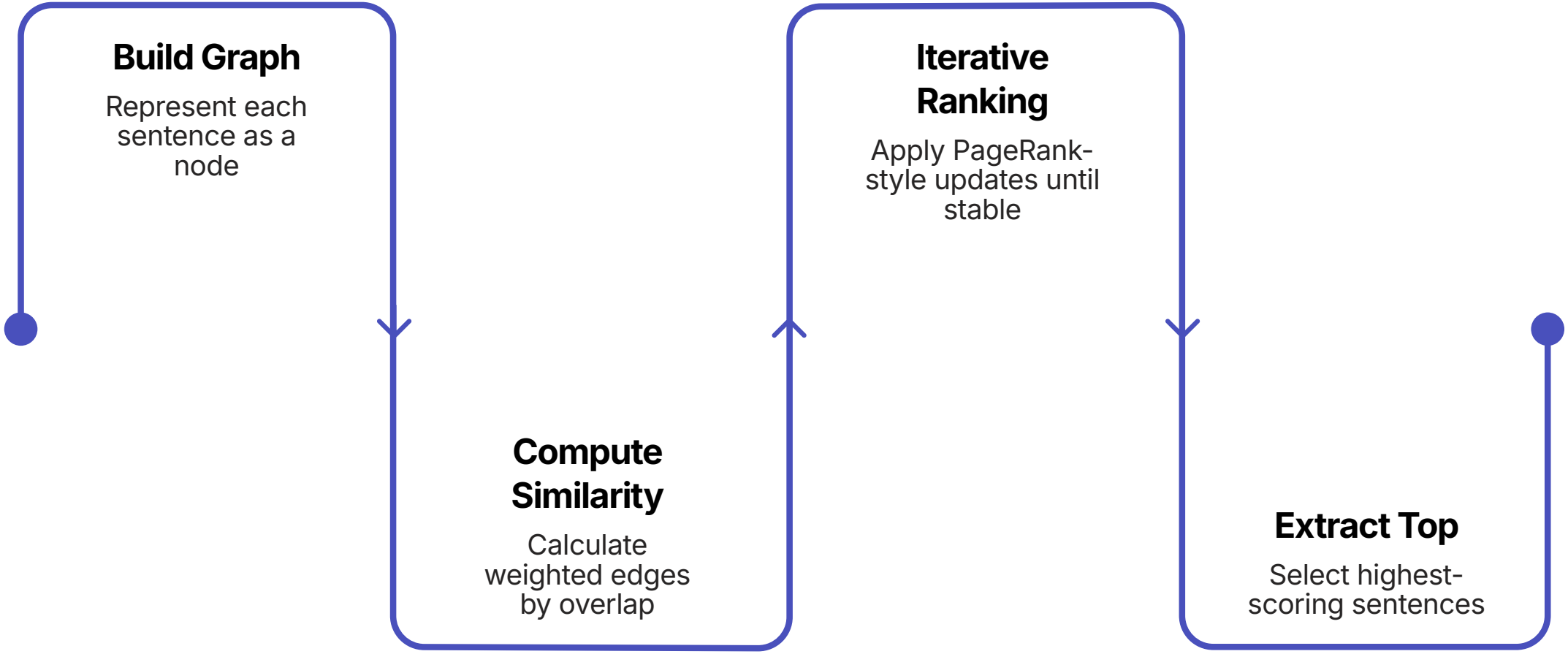
3

Graph-Based

TextRank treats sentences as nodes; important sentences are similar to many other important sentences

TextRank Algorithm Deep Dive

TextRank applies PageRank-style graph algorithms to text. Each sentence becomes a node, and edges represent similarity between sentences based on word overlap.



The algorithm iteratively updates sentence scores until convergence, with important sentences being those highly connected to other important sentences—a recursive definition similar to PageRank for web pages.

ABSTRACTIVE METHODS

Abstractive Summarization

Abstractive approaches use sequence-to-sequence models (the same architecture as neural MT) to generate summaries. The model learns to paraphrase, compress, and synthesize information into new sentences.

Advantages

More natural language, better compression, can reformulate and combine ideas

Modern Models

- BART: Denoising autoencoder pre-trained on document corruption tasks
- T5: Text-to-text framework treating all NLP as sequence transduction
- PEGASUS: Pre-trained specifically for summarization using gap sentences

The Hallucination Problem

Abstractive models can generate fluent, compelling text that contains factual errors or entirely fabricated information—a phenomenon called hallucination.

Since the model generates freely rather than copying source sentences, it may introduce plausible-sounding but incorrect facts, dates, names, or relationships. This remains an active research challenge, with approaches including retrieval-augmented generation and fact-checking modules.

t mention of artificial intelligence
imes was in a ~~February 19, 1950~~
1950 article titled ~~"Thinking Mac~~
n' is Seen in 'Thinking Machines.'
~~Walter Sullivan,~~ reported on a me
~~an Association for the Advancem~~
here a number of scientists discu
of creating machines that could t

EVALUATION

ROUGE: Summarization Evaluation

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) measures how many facts from human-written reference summaries appear in the machine-generated summary.

ROUGE-N

Overlap of n-grams between candidate and reference summaries

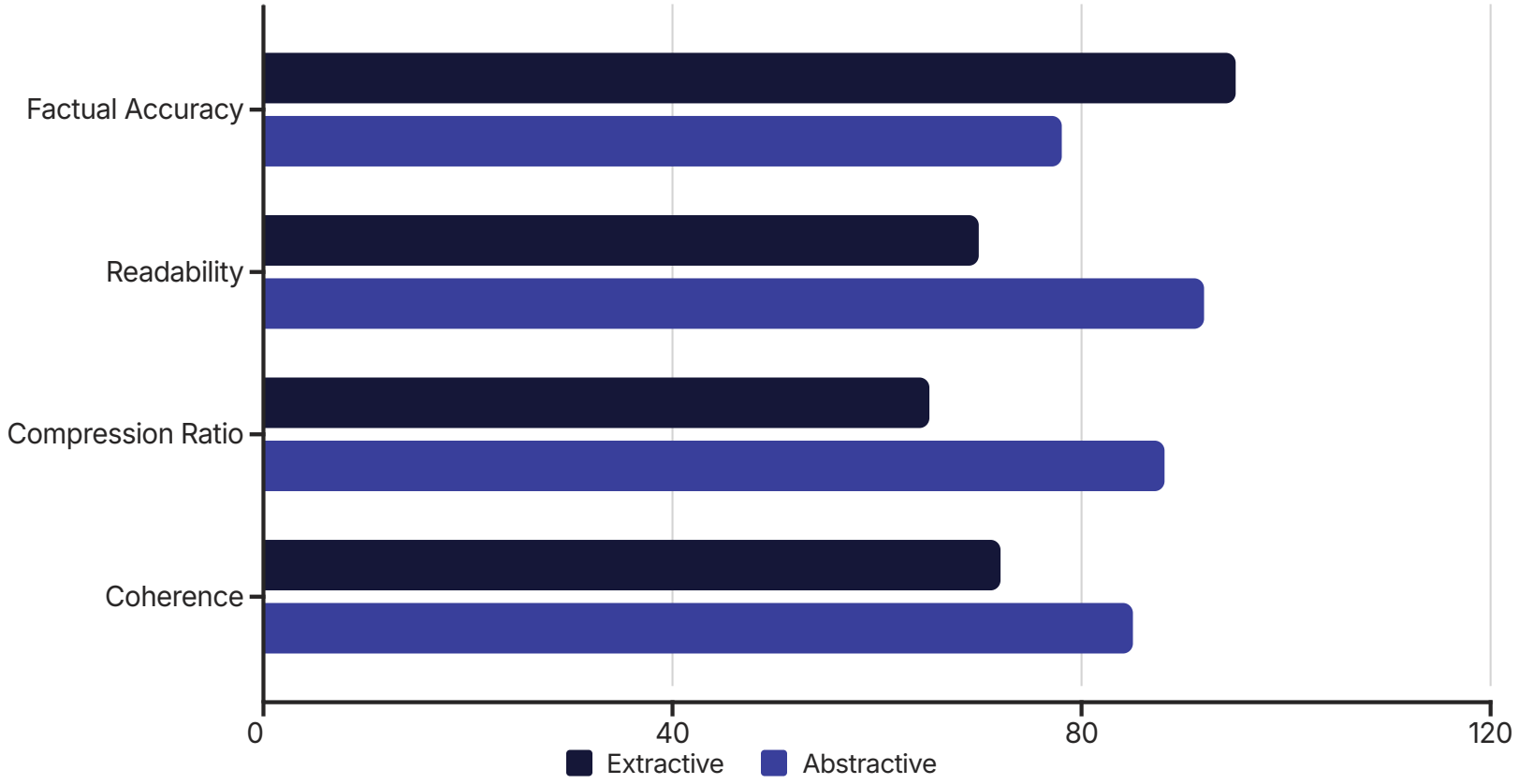
ROUGE-L

Longest common subsequence, capturing sentence-level structure

ROUGE-S

Skip-bigram co-occurrence allowing for flexible word ordering

Extractive vs. Abstractive Comparison

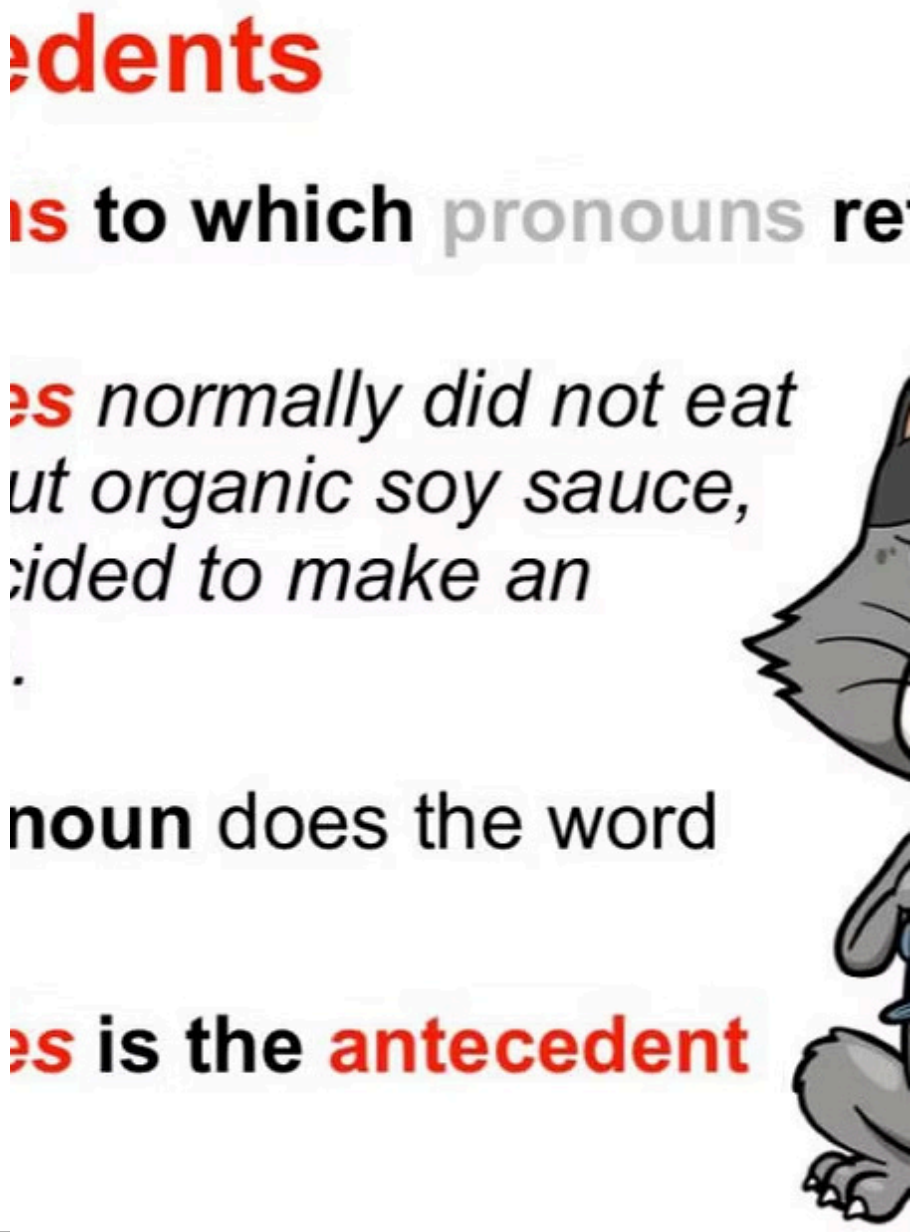


Extractive methods guarantee factual accuracy but produce less fluid summaries. Abstractive methods create more natural language but risk introducing errors.

LINGUISTIC CHALLENGE

Coreference Resolution

Coreference resolution identifies when multiple expressions in a text refer to the same real-world entity. This is essential for understanding "who did what" in narratives.



Coreference: A Concrete Example

> "**The CEO** entered the room. **She** looked tired. After **the meeting**, **it** was clear **the executive** needed rest."

Cluster 1: Person

The CEO → She → the executive

Cluster 2: Event

the meeting → it

Successfully linking these mentions enables downstream tasks like summarization to maintain coherent entity references.

Why Coreference Matters

Without coreference resolution, automatic summarization produces confusing output with unclear pronoun references. Question answering systems fail to track entities across sentences. Information extraction misses important relationships.

Summarization

If the summarizer doesn't know "he" refers to "the scientist," extracted sentences become incoherent

Question Answering

"What did she discover?" requires linking "she" to the correct person mentioned earlier

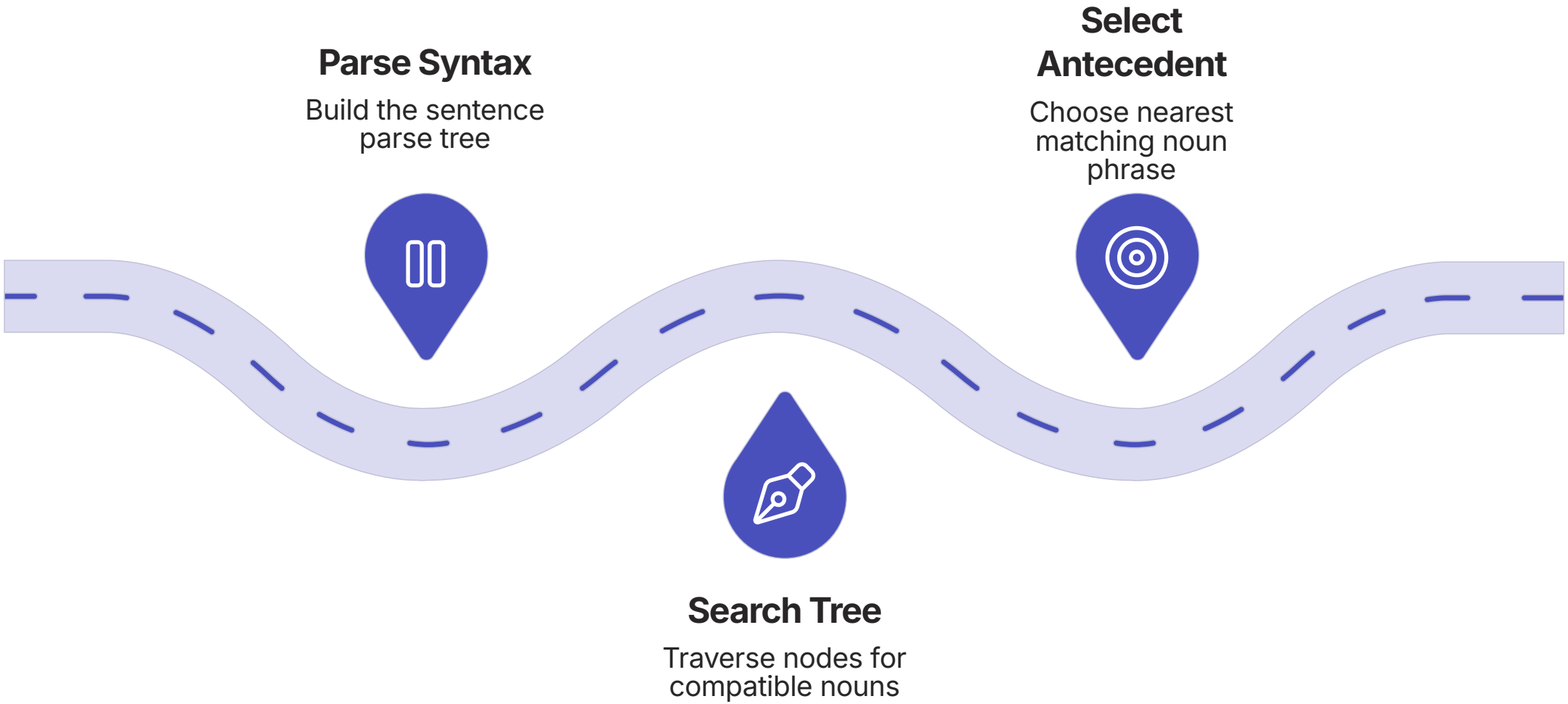
Information Extraction

Building knowledge graphs requires merging all mentions of the same entity

TRADITIONAL APPROACH

Hobbs' Algorithm

Hobbs' Algorithm (1978) was an influential rule-based approach that used syntactic parse trees to resolve pronouns. The algorithm performs a tree search to find the nearest noun phrase matching the pronoun's gender and number.



While historically important, rule-based methods struggle with complex cases requiring world knowledge or semantic understanding.

Modern Neural Coreference

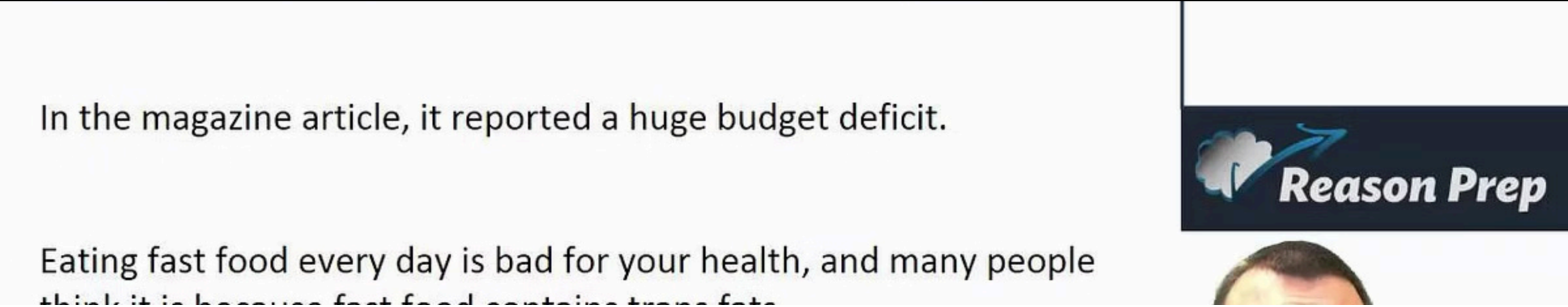
Contemporary approaches use machine learning to cluster mentions based on learned features. Neural models consider gender, number, semantic similarity, syntactic distance, and contextual embeddings.

Feature Types

- String matching and head word agreement
- Gender and number compatibility
- Semantic role similarity
- Syntactic distance in parse tree
- Contextual word embeddings (BERT)

Popular Models

End-to-end neural models like SpanBERT and CorefQA achieve state-of-the-art results by considering all possible mention spans and learning to cluster them jointly.



The Ambiguity Challenge

Many coreference cases require world knowledge or deep semantic understanding that remains challenging for current systems.

> "The dog chased the cat, which was faster."

Does "which" refer to the dog or the cat? Machines struggle to leverage world knowledge (cats are typically faster than dogs) or context clues to resolve such ambiguities correctly.

GROUP PROJECT

Course Project Launch

The final project provides an opportunity to apply course techniques to a real-world NLP problem. Work in small groups to design, implement, and evaluate a complete NLP system.

01

Form Teams

Groups of 3-4 students with complementary skills

02

Choose Topic

Select a task and dataset from provided options or propose your own

03

Submit Proposal

1-page document due next session outlining your approach

04

Implementation

Build and evaluate your system over the remaining weeks

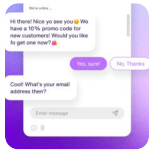
Project Ideas & Datasets

Sentiment Analysis

Build a classifier for Twitter data using the Sentiment140 dataset with 1.6M labeled tweets

News Summarization

Implement TextRank or neural summarization using the CNN/Daily Mail dataset



FAQ Chatbot

Create a domain-specific question-answering system using retrieval or generation

Named Entity Recognition

Train a sequence labeler on CoNLL-2003 to identify people, organizations, locations

Project Proposal Requirements

Your 1-page proposal should clearly articulate the problem, approach, and evaluation plan. This helps ensure your project scope is appropriate and feasible within the time constraints.

1	**Task Definition** What problem are you solving? What is the input and desired output?
2	**Dataset Description** What data will you use? How large is it? What preprocessing is needed?
3	**Model Architecture** What techniques will you apply? Why are they appropriate for this task?
4	**Evaluation Plan** What metrics will you use? What baseline will you compare against?

</> PRACTICAL ASSIGNMENT

Assignment 13: Exploring MT with APIs

Gain hands-on experience with modern neural machine translation systems through experimentation with pre-trained models and analysis of their behavior.

1 Back-Translation Experiment

Use googletrans or Hugging Face MarianMT to translate complex English sentences to French, then translate back to English. Analyze meaning preservation and idiom handling.

2 Pre-trained Model Demo

Load Helsinki-NLP/opus-mt-en-fr from Hugging Face and experiment with various sentence types. Document strengths and failure modes.

Assignment 14: Summarization & Coreference

- **Extractive Summarization**

  Implement TextRank using the sumy or gensim library on a news article. Evaluate whether the top 3 sentences capture the most important information.

- **Coreference Resolution**

  Use the spacy-coref extension or AllenNLP to resolve pronouns in a multi-sentence paragraph. Map out the coreference clusters.

- **Abstractive Comparison**

  Use a T5 or BART model via Hugging Face to summarize the same article. Compare readability, coherence, and factual accuracy with the extractive version.

- **Project Proposal**

  Submit your 1-page project proposal following the required format.

☆ KEY TAKEAWAYS

Session Summary

MT Evolution

Machine translation progressed from brittle rule-based systems to data-driven statistical models to powerful neural seq2seq architectures with attention

Summarization Approaches

Extractive methods guarantee factual accuracy while abstractive methods produce more natural language at the risk of hallucination

Coreference Importance

Resolving entity mentions across text is fundamental for coherent understanding in downstream NLP tasks

Practical Application

Modern pre-trained models make advanced NLP accessible through simple APIs, enabling rapid prototyping and deployment